

# Linux and Your Machine's Memory and Storage

A 75 minute walkthrough for the next class

---

Every technology in Chapters 6 and 7 is sitting inside the machine in front of you right now. The DIMMs, whether they carry error correction, whether a given drive spins or not, and which scheduler Linux picked for it, are not historical detail. They are things a running Linux system reports about itself directly. Today we look at four of them on your own machine, then watch a fifth, a RAID mirror, built and broken live on one shared screen.

**Before this session, demo machine only.** Install `mdadm` on the machine you plan to demo from. Students do not need this package, only Part 5 uses it, and that part runs on one machine. On Arch, that is `sudo pacman -S mdadm`.

**Two screens, one system.** For Parts 1 through 4, run every command on your own terminal at the same time the instructor runs it. The exact numbers will differ, sometimes a lot, but the shape of what you are looking at, a DIMM listing, an error correction type, a rotational flag, a scheduler name, will not. For Part 5, everyone gathers around one machine instead, since that part is built to be watched and reasoned about together, not repeated four times.

**If a command is missing.** If `dmidecode` prints “command not found” or a permission error, the setup checklist sent before class was not completed for this machine. See the appendix at the end of this handout for the install command for your distribution, or follow along on the instructor’s screen for that step and catch up afterward.

## 1. DIMM inventory, live

Chapter 6 opens with the memory cell itself, Figure 6.1, and Table 6.1’s family of semiconductor memory types. The physical DIMMs installed in this machine are an instance of that table, and `dmidecode` reads them straight from the motherboard’s firmware.

```
sudo dmidecode --type 17
```

Each block in the output is one memory slot. Size and Type tell you how much is installed and which DDR generation it is. Speed, Manufacturer, and Part Number identify the exact module. An empty slot reports “No Module Installed” rather than an error, that is expected on a machine that is not fully populated.

**Discussion.** Compare your Type field with the person next to you and with the instructor’s screen. Chapter 6’s Table 6.4 lists DDR1 through DDR4 by clock rate and transfer rate. Find your generation in that table and say out loud what its actual transfer rate is, not just its name.

## 2. Error correction, live

Table 6.2 shows that adding error correction costs real bits, a longer word for the same data, in exchange for the ability to detect and often fix a flipped bit. Whether that trade was made on this machine is a single firmware field.

```
sudo dmidecode --type 16 | grep -A1 "Error Correction"
```

**Discussion.** Most laptops report “None.” Consumer memory generally skips error correction entirely, while server memory usually reports “Single-bit ECC” or better. That split is the tradeoff Table 6.2 describes made real, extra check bits and extra memory controller logic cost money and a small amount of performance, and most machines are not built to spend that cost.

Now check whether the kernel is tracking any correctable or uncorrectable errors live.

```
ls /sys/devices/system/edac/mc/
```

**Nothing under mc/?** This is the expected result on most laptops, and it is not a missing setup step. The EDAC subsystem only has something to report when the memory controller itself supports error correction. If Error Correction Type came back “None” in the previous step, this being empty is the same fact confirmed a second way, from the running kernel rather than from firmware.

## 3. Spinning or solid state, live

Chapter 7 separates external memory by access method and mechanism, magnetic disk with a moving head in Figure 7.2, an SSD with no moving parts at all in Figure 7.8. Linux exposes that same distinction as one column.

```
lsblk -d -o name,rota,size,model
```

**Discussion.** A 1 in the ROTA column means an actual spinning platter, the mechanism behind Table 7.2’s disk performance parameters. A 0 means solid state, no seek time, no rotational latency, the architecture in Figure 7.8. Say out loud which of your own drives are which before comparing with the instructor’s screen.

## 4. The I/O scheduler, live

A scheduler decides what order pending read and write requests are sent to a drive in. That decision only matters because of the mechanism question just answered.

```
for d in /sys/block/*/queue/scheduler; do echo "$d"; cat "$d"; done
```

The active scheduler for each device is the one shown in square brackets.

**Discussion.** A spinning disk pays a real seek and rotation cost that depends on where the read and write head currently sits, so reordering requests to minimize head movement, the job of schedulers such as **bfq** or the classic elevator algorithm, saves real time. Solid state storage has no head and no seek cost, so Linux commonly assigns it **none** or **mq-deadline** instead, since there is nothing left

to optimize by reordering. Match each device's scheduler against its ROTA value from the previous step. If the two do not line up the way this discussion predicts, that mismatch is worth asking about rather than skipping past.

## 5. RAID Level 1, built and broken

**How this section runs.** One machine, one demo, everyone watching. Before every command marked **Predict.** below, each of you states out loud what you expect to happen and why, before the instructor runs it. Trade off who is at the keyboard between commands, and whoever is not typing has the job of explaining back to the group what the output actually showed before the class moves on.

Table 7.3 lists RAID Level 1 as pure mirroring, every byte written to one member is written in full to every other member. Today's demo builds a real Level 1 array out of two plain files acting as virtual disks, so nothing on the demo machine's real drives is touched.

```
sudo mkdir -p /root/raid-demo
sudo fallocate -l 256M /root/raid-demo/disk1.img
sudo fallocate -l 256M /root/raid-demo/disk2.img
ls -lh /root/raid-demo/
```

**Discussion.** Two ordinary 256 MB files, nothing else. Nothing about them is special yet, and that is the point, RAID does not require dedicated hardware, only something the kernel can address as a block device.

**What “loop” means here.** A loop device lets the kernel treat an ordinary file as if it were a physical disk. The name comes from loopback, the same idea behind the loopback network interface at 127.0.0.1, instead of sending data out to a real controller, the kernel loops it back around into the regular file system underneath. This is also how mounting a downloaded ISO file works without burning it to anything first. The numbers in `/dev/loop0` and `/dev/loop1` do not mean anything on their own, they are just the first two loop slots the kernel had free to hand out, the same way `sda` and `sdb` simply number physical disks in the order Linux found them.

```
sudo losetup --find --show /root/raid-demo/disk1.img
sudo losetup --find --show /root/raid-demo/disk2.img
losetup -a
```

Note the two device names `losetup` prints, most likely `/dev/loop0` and `/dev/loop1`. The `losetup -a` line confirms exactly which file backs each device. Those two devices stand in for two physical disks for the rest of this section.

**Predict.** Two independent files are about to become one mirrored array. Before running the next command, say what you expect the two loop devices to contain once this finishes, identical data, split data, or something else.

**A prompt about a write-intent bitmap.** This command may ask whether to enable a write-intent bitmap, a feature that speeds up recovery after an unclean shutdown by tracking which regions changed since the last sync. Answer no, or press Enter, since N is already the default. It costs a small amount of write performance and has nothing to do with what this demo is teaching, so there is no need to enable it today.

**A prompt about metadata and boot loaders.** This command may also warn that the array's metadata may not be suitable as a boot device, then ask whether to continue creating the array. Answer y here. This array is never used to boot anything, it only ever holds a demo file, so the warning does not apply. Answering N or pressing Enter aborts the whole command instead of creating the array, which is not what you want.

```
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/loop0 /dev/loop1
cat /proc/mdstat
lsblk
```

**Discussion.** `/proc/mdstat` now shows `md0` built from both loop devices, each marked as an active member, and `lsblk` draws the same fact as a tree, `md0` sitting on top of both loop devices. This matches Figure 7.6's Level 1 diagram directly, two disks, one logical array, full duplication rather than the striping Level 0 uses.

```
sudo mkfs.ext4 /dev/md0
sudo mkdir -p /mnt/raid-demo
sudo mount /dev/md0 /mnt/raid-demo
echo "Chapter 7, Level 1, written once and stored twice" | sudo tee /mnt/raid-demo/
    message.txt
df -h /mnt/raid-demo
ls -l /mnt/raid-demo/
```

**Discussion.** `df` shows a real filesystem mounted with real free space, and `ls` shows the file sitting inside it, exactly as it would on any ordinary disk. Nothing about how this looks from inside the mount point reveals that two files are standing in for the storage underneath.

The operating system was only ever told to write this sentence once, to `/dev/md0`. What actually happened underneath is worth seeing directly, not just trusting.

```
sudo grep -a "Chapter 7" /root/raid-demo/disk1.img
sudo grep -a "Chapter 7" /root/raid-demo/disk2.img
```

**Discussion.** Both plain files contain the exact sentence, even though only one write command was ever issued. Mirroring is not a promise the array makes about itself, it is a physical fact sitting in two ordinary files right now, readable with a command that has never heard of RAID.

**Predict.** One of the two loop devices is about to disappear from the array on purpose. Before running the next command, say what you expect to happen to `message.txt`, and why that follows from what Level 1 actually does with every write.

```
sudo mdadm /dev/md0 --fail /dev/loop0
cat /proc/mdstat
cat /mnt/raid-demo/message.txt
```

**Discussion.** `/proc/mdstat` now shows the array degraded, with `loop0` marked faulty, yet the file reads back whole. That is the entire point of Table 7.3's Data Availability column for Level 1, a single member failing costs redundancy, not data. Ask the group what would have happened here if this had been a Level 0 array instead, where Table 7.3 lists availability as "lower than single disk."

```
sudo grep -a "Chapter 7" /root/raid-demo/disk1.img
sudo grep -a "Chapter 7" /root/raid-demo/disk2.img
```

**Discussion.** The sentence is still sitting inside `disk1.img`, the member just marked faulty. Failing a device tells the array to stop trusting it, it does not erase what was there, which is also why a failed drive can sometimes be recovered rather than replaced.

**Clean this up before the room empties.** This demo only touched loop devices and files under `/root/raid-demo`, nothing on a real partition, but it should still be torn down so the next class starts from a clean machine.

```
sudo umount /mnt/raid-demo
sudo mdadm --stop /dev/md0
sudo losetup -d /dev/loop0
sudo losetup -d /dev/loop1
sudo rm -rf /root/raid-demo /mnt/raid-demo
ls /root/raid-demo/
```

The final `ls` should report no such file, confirming the teardown actually happened rather than just trusting the previous command succeeded.

## 6. Suggested timing

Table 1: A 75 minute breakdown.

| Block                                  | Time       |
|----------------------------------------|------------|
| Setup check, stragglers catch up       | 5 minutes  |
| DIMM inventory, live                   | 9 minutes  |
| Error correction, live                 | 7 minutes  |
| Spinning or solid state, live          | 7 minutes  |
| The I/O scheduler, live                | 7 minutes  |
| RAID Level 1, built and broken         | 35 minutes |
| Wrap up and connection to next session | 5 minutes  |

## 7. Wrap up

Everything in Parts 1 through 4 only read information off the machine, nothing was changed or configured. Part 5 changed something on purpose, and then undid it, on files built to be disposable. The same rotational question from Part 3 will have exactly one answer on the Raspberry Pi units next time, since they carry no spinning disk at all, only a flash based SD card as external memory, so you will already know what to expect before you even ask.

## 8. Reference, concept to command

Table 2: What each command reveals, and where it appears in the text.

| Concept                              | Command                                                 | Textbook location                 |
|--------------------------------------|---------------------------------------------------------|-----------------------------------|
| <b>DIMM inventory</b>                | <code>sudo dmidecode --type 17</code>                   | Ch 6, Table 6.1, Fig 6.2          |
| <b>Error correction type</b>         | <code>sudo dmidecode --type 16</code>                   | Ch 6, Table 6.2                   |
| <b>Live ECC error counts</b>         | <code>ls /sys/devices/system/edac/mc/</code>            | Ch 6, Fig 6.9 to 6.11             |
| <b>Rotational versus solid state</b> | <code>lsblk -d -o name,rota,size,model</code>           | Ch 7, Fig 7.2, Fig 7.8            |
| <b>I/O scheduler choice</b>          | <code>cat /sys/block/*/queue/scheduler</code>           | Ch 7, disk performance parameters |
| <b>RAID Level 1 mirror</b>           | <code>sudo mdadm --create /dev/md0 --level=1 ...</code> | Ch 7, Table 7.3, Fig 7.6          |

## 9. Appendix, install commands

If you did not complete the setup checklist before class, install `dmidecode` now.

**Table 3:** Install commands by distribution.

| Distribution       | Command                                           |
|--------------------|---------------------------------------------------|
| <b>Arch Linux</b>  | <code>sudo pacman -S dmidecode</code>             |
| <b>Artix Linux</b> | <code>sudo pacman -S dmidecode</code>             |
| <b>Gentoo</b>      | <code>sudo emerge --ask sys-apps/dmidecode</code> |
| <b>Void Linux</b>  | <code>sudo xbps-install -S dmidecode</code>       |
| <b>Xubuntu</b>     | <code>sudo apt install dmidecode</code>           |